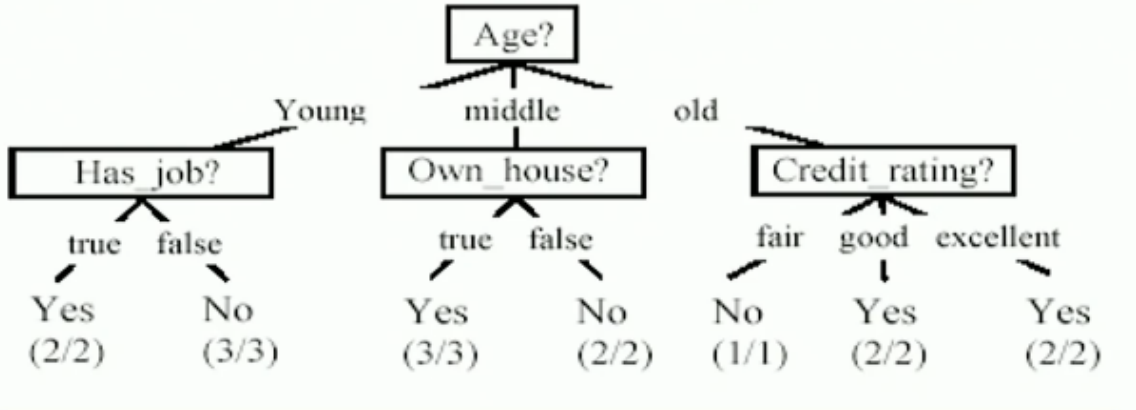
- Goal: fare un modello di classificazione che può predire una classe di una nuova istanza
  - Correlare attributi => classe
  - Noi diamo in input dati labellati, siamo nel supervisioned
- A seconda dell'output, può essere sia usato per la regressione che regressione
  - Classificazione: che tipo di classe/approvato
  - Regressione: valore numerato
    - In questo caso, se abbiamo attributi con valori viene sempre lasciato nell'insieme A, e mai tolto
- Sono molto efficienti e buon performanti, sopratutto in linguaggio tabellare (non naturale)
- Prob:
  - Funziona molto bene sui dati categorici
  - Si può passare da albero a regole facilmente
- Pro/Contro:
  - Facciamo induzione sui dati per creare alberi
  - In ogni nodo troviamo una condizione logica sulla feature che ci dice dove andare
  - E quando arriviamo ad una foglia è la classe
  - Si può facilmente mancare dei valori
  - Se abbiamo pochissimi elementi, potrebbe creare problemi di classificazione
    - E se vogliamo dare più importanza a certi attributi rispetto agli altri



  - E' come se facessimo bruteforcing? Fai tutti i casi
    - Non esiste 1 solo albero, potremmo cambiare es. nodo iniziale, però gli alberi sono simili
    - Noi vogliamo un albero accurato, aka che sia sempre X/X e non X/Y
      - Supporto = quanti elementi del dataset ho usato
      - Confidenza = Quanti elementi sono classificati correttamente
    - Spesso si prova diversi nodi per ottimizzare i X/X attraverso delle euristiche

        - Si può trasformare in una serie di decisioni logiche, regole (tanti if)
        - per via della struttura, sappiamo anche sapere il perchè, non è una black box
        - L'obiettivo è avere delle foglie con l'impurità più bassa possibile
    - Problema:
        - e se le decisioni non fossero categorie o si/no ma dei numeri?
            - In questo caso si fa con <= e >, però nota che gli attributi li possiamo considerare più volte
        - E' facile andare in overfitting se stiamo troppo attaccati ai dati di training, succede se facciamo troppi casi con troppe variabili
          Fix:
            - Pre-pruning: si tagliano parti di alberi durante l'algoritmo stesso se notiamo che stiamo andando troppo nel dettaglio
            - Post-pruning: tagliamo rami o sotto alberi da un albero già completo
- Algoritmo:
    - E' un algoritmo divide e timpera, atto ricorsivamente
    - E' costruito da sopra al sotto
    - Scegliamo l'attributo per la partizione (es. Age, has_job) attraverso una funzione information gain
    - Input:
        - Insieme D da partizionare (dataset)
        - Attributi A che possiamo partizionare
        - T risultato algoritmo (le foglie delle classi)
    - Scritto sottile sottile:
      if siamo in un branch puro, aka in D abbiamo tutte classi uguali C
      allora aggiungi foglia C all'albero T
      Elif non abbiamo nessun attributo da considerare, cioè A è vuoto
      Allora aggiungiamo foglia con la classe più frequente di D
      Else:
      p0 = ImpurityVal(D) => Stimiamo quanto D è non puro
      for each attributo a in A allora
      pi = impurityVal(a, D)
      Poi selezioniamo l'attributo che riduce di più l'impurità attraverso p0
      if se l'impurità è troppo piccola, allora aggiungi una nuova foglia con cp più frequente (non vogliamo fare overfitting, pre-pruning)
      else
      Per ogni gruppi dell'attributo Ap scelto, creiamo un gruppo (età giovane, adulto, anziana)
      for each gruppo creato
      Richiama l'algoritmo Dj, togliamo l'attributo del gruppo, e ripassiamo T

- La maggior parte degli algoritmo di decision tree variano a seconda dell'impurityVal